CS336 作业 1（基础篇）：构建一个 Transformer 语言模型

版本 1.0.6
CS336 Staff
2025 年春季

> 说明：本文档为 `cs336_spring2025_assignment1_basics.pdf` 的中文翻译整理版。
> 为了便于阅读，我保留了原有章节结构、题目编号、公式和关键代码接口，并去掉了 PDF 抽取后的分页噪音。
> 参考文献条目保留英文原貌。

1. 作业概览

在本次作业中，你将从零实现训练一个标准 Transformer 语言模型（LM）所需的全部组件，并训练若干模型。

你将实现的内容

1. Byte-pair encoding（BPE）分词器（第 2 节）
2. Transformer 语言模型（第 3 节）
3. 交叉熵损失函数与 AdamW 优化器（第 4 节）
4. 训练循环，支持序列化与加载模型/优化器状态（第 5 节）

你将实际运行的内容

1. 在 TinyStories 数据集上训练一个 BPE 分词器。
2. 用你训练好的分词器处理数据集，把文本转换为整数 ID 序列。
3. 在 TinyStories 上训练一个 Transformer LM。
4. 使用训练好的 Transformer LM 生成样本并评估困惑度。
5. 在 OpenWebText 上训练模型，并把你得到的困惑度提交到排行榜。

允许使用的内容

我们希望你尽可能从零实现这些组件。具体来说，除了以下内容外，你**不能**使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中现成的定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类，例如 `Module`、`ModuleList`、`Sequential` 等
- `torch.optim.Optimizer` 基类

除此之外，你可以使用任何其他 PyTorch 定义。
如果你不确定某个函数或类是否允许使用，可以在 Slack 上提问。一个基本判断标准是：它是否破坏了本作业“从零实

现”的精神。

关于 AI 工具的声明

可以向 ChatGPT 之类的大模型咨询：

- 低层次的编程问题
- 高层次的语言模型概念问题

但**禁止**直接使用它来替你解题。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全功能（例如 Cursor Tab、GitHub Copilot）。
非 AI 自动补全（例如函数名自动补全）当然可以使用。
我们的经验是，AI 自动补全会显著削弱你对内容的深入理解。

代码结构说明

本次作业的全部代码与 handout 都在 GitHub 上：

```
github.com/stanford-cs336/assignment1-basics
```

请先 `git clone` 这个仓库。若后续有更新，我们会通知你执行 `git pull` 。

仓库中几个关键文件/目录

1. `cs336_basics/*`
   这是你编写代码的主要位置。这里默认没有现成实现，你可以完全从零组织代码。
2. `adapters.py`
   这里定义了一组你的代码必须暴露出来的接口。
   对于每一项功能（例如 scaled dot-product attention），你需要在相应的适配函数中调用你自己的实现。
   注意：你对 `adapters.py` 的修改不应包含实质性逻辑；它只是胶水层。
3. `test_*.py`
   这些是你必须通过的测试文件，例如 `test_scaled_dot_product_attention` 。
   测试会通过 `adapters.py` 中定义的钩子调用你的实现。
   不要修改测试文件。

如何提交

你需要在 Gradescope 提交以下内容：

- `writeup.pdf` ：回答所有书面题。请使用排版良好的形式提交。
- `code.zip` ：包含你编写的所有代码。

如果你要提交排行榜成绩，请向以下仓库提交一个 PR：

`github.com/stanford-cs336/assignment1-basics-leaderboard`

详细说明见该排行榜仓库中的 `README.md` 。

数据集从哪里获取

本次作业会使用两个经过预处理的数据集：

- TinyStories [Eldan and Li, 2023]
- OpenWebText [Gokaslan et al., 2019]

这两个数据集都被存成单个大型纯文本文件。
如果你是跟着课程环境做作业，可以在任意非 head node 机器的 `/data` 下找到它们。
如果你是在本地跟做，可以根据 `README.md` 中的命令下载。

低资源 / 降规模提示：总则

在课程 handout 中，我们会不断给出一些关于“资源不足时怎么继续推进”的建议。
例如：

- 缩小数据集或模型规模
- 在 MacOS 集成 GPU 或 CPU 上运行训练

这些“低资源提示”会以蓝框形式出现。即便你能使用课程机器，这些建议通常也能帮助你更快迭代并节省时间，因此仍然值得阅读。

低资源 / 降规模提示：在 Apple Silicon 或 CPU 上完成作业 1

使用 staff 参考实现时，我们可以在 Apple M3 Max（36GB RAM）上：

- 使用 Metal GPU（MPS）在 5 分钟内训练出能生成还算流畅文本的小型 LM
- 或使用 CPU 在约 30 分钟内完成训练

如果这些硬件名词你不熟悉，也没关系。你只需要知道：

> 如果你的笔记本比较新、实现正确且足够高效，那么你完全可以训练出一个能生成简单儿童故事的小语言模型。

后文会进一步解释在 CPU 或 MPS 上需要做哪些调整。

2. Byte-Pair Encoding（BPE）分词器

在作业的第一部分，你将训练并实现一个**字节级 byte-pair encoding（BPE）分词器** [Sennrich et al., 2016; Wang et al., 2019]。

更具体地说：

- 我们会把任意 Unicode 字符串表示成**字节序列**
- 在这个字节序列上训练 BPE 分词器
- 随后再用该分词器把文本编码成 token（整数序列），供语言模型使用

2.1 Unicode 标准

Unicode 是一个文本编码标准，它把字符映射为整数码点（code point）。

截至 Unicode 16.0（2024 年 9 月发布），该标准定义了：

- 154,998 个字符
- 覆盖 168 个书写系统

例如：

- 字符 `"s"` 的码点是 115（通常记作 `U+0073` ）
- 字符 `"牛"` 的码点是 29275

在 Python 中：

- `ord()` ：把单个 Unicode 字符转成整数码点
- `chr()` ：把整数码点转回 Unicode 字符串

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

题目（unicode1）：理解 Unicode（1 分）

(a) `chr(0)` 返回的是哪个 Unicode 字符？

交付内容：一句话回答。

(b) 该字符的字符串表示（ `__repr__()` ）与其打印表示有什么不同？

交付内容：一句话回答。

(c) 当这个字符出现在文本中时会发生什么？你可以在 Python 解释器里试试：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付内容：一句话回答。

2.2 Unicode 编码

Unicode 标准定义了“字符到码点”的映射，但如果直接在 Unicode 码点上训练分词器，会很不现实，因为：

- 词表会非常大（约 15 万项）
- 非常稀疏，很多字符极其少见

因此，我们会使用一种 **Unicode 编码（Unicode encoding）**，把一个 Unicode 字符转换为一串字节。

Unicode 标准定义了三种主流编码：

- UTF-8
- UTF-16
- UTF-32

其中，UTF-8 是互联网的主流编码（超过 98% 网页使用它）。

Python 中的 UTF-8 编码与解码

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 12
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

关键点：

- 一个字节不一定对应一个 Unicode 字符
- 经过 UTF-8 编码后，字符序列会变成 0 到 255 的字节值序列
- 这使得词表大小固定为 256，便于处理

因此，如果采用字节级分词：

> 我们永远不会遇到 OOV（词表外）字符，因为任何输入文本都能表示成 0 到 255 之间的字节序列。

题目（unicode2）：Unicode 编码（3 分）

(a) 相比 UTF-16 或 UTF-32，为什么我们更倾向于在 UTF-8 编码后的字节上训练分词器？
你可以比较不同输入字符串在这些编码下的输出。

交付内容：1 到 2 句话回答。

(b) 考虑下面这个错误实现，它试图把 UTF-8 字节串解码为 Unicode 字符串：

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

为什么它是错的？请给出一个会产生错误结果的输入字节串。

交付内容：给出一个输入示例，并用一句话解释为什么该函数错误。

(c) 给出一个无法解码成任何 Unicode 字符的两字节序列。

交付内容：给出一个示例，并用一句话解释。

2.3 子词分词（Subword Tokenization）

字节级分词虽然避免了 OOV 问题，但也会造成序列非常长：

- 一个 10 个词的句子，在词级模型里可能只有 10 个 token
- 在字节级模型里可能会变成 50 个甚至更多 token

这会导致：

- 训练更慢
- 每步计算开销更大
- 更长的依赖关系更难建模

子词分词介于：

- 词级分词
- 字节级分词

之间，是一种折中方案。

字节级分词器的词表只有 256 个条目；
子词分词器则会使用更大的词表，以换取对输入序列更好的压缩。

例如，如果字节串 `b'the'` 在训练语料中频繁出现，那么把它放进词表后：

- 原本 3 个 token 的字节序列
- 就可以压缩成 1 个 token

那么这些子词单位如何选择？

Sennrich 等人 [2016] 提出使用 **BPE（Byte-Pair Encoding）**：

- 每轮找到最常见的相邻字节对
- 用一个新 token 替换它
- 不断迭代

因此：

> BPE 分词器会把那些能最大化压缩率的字节组合加入词表。

本作业中，你将实现的是**字节级 BPE 分词器**，词表项可以是：

- 单个字节
- 或多个字节合并而成的字节序列

构造这个词表的过程就叫做训练 BPE 分词器。

2.4 BPE 分词器训练

BPE 训练包含三个主要步骤。

1. 初始化词表

分词器词表是一个从“字节串 token”到“整数 ID”的映射。
由于我们训练的是**字节级 BPE**，初始词表就是所有单字节：

- 共 256 个可能字节值
- 所以初始词表大小为 256

2. 预分词（Pre-tokenization）

理论上，你可以直接在整个语料上统计相邻字节对频率并开始合并。
但这样会有两个问题：

1. 每合并一次都要重新扫整个语料，计算代价非常高
2. 容易把仅在标点上不同的 token 分得很碎，例如 `dog!` 与 `dog.`

为此，我们先对语料做一层粗粒度预分词。

你可以把它理解成：

- 先把语料切成较大的 pre-token
- 再在每个 pre-token 内统计字符对频率

例如某个 pre-token `'text'` 出现了 10 次，那么统计 `'t'` 和 `'e'` 的相邻频率时，就可以直接加 10，而不必再次逐字扫描原语料。

由于我们训练的是字节级 BPE，每个 pre-token 最终仍会被表示为 UTF-8 字节序列。

Sennrich 等人 [2016] 原始 BPE 的预分词只是简单按空白拆分： `s.split(" ")` 。
而本作业要求你使用 GPT-2 使用的基于正则的 pre-tokenizer：

```
PAT = r""""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

示例：

```
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

在实际代码中，建议你使用 `re.finditer` ，这样无需显式保存所有预分词结果，就能构建 pre-token 到频次的映射。

3. 计算 BPE merges

现在我们已经把输入文本表示成：

- 一系列 pre-token
- 每个 pre-token 又是 UTF-8 字节序列

于是可以开始执行 BPE merge。

高层过程如下：

1. 统计所有相邻字节对的出现次数
2. 找到频率最高的字节对 `("A", "B")`
3. 把其每次出现都合并成新 token `"AB"`
4. 把这个新 token 加入词表
5. 重复上述过程

因此最终词表大小为：

- 初始字节词表大小 256
- 加上所有 merge 产生的新 token

为了提高效率，本作业中**不考虑跨 pre-token 边界的 pair**。

如果频率并列，则使用如下规则打破平局：

> 选择字典序（lexicographically）更大的 pair。

例如若以下 pair 频率相同：

```
[("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")]
```

那么应该选择：

```
('BA', 'A')
```

特殊 token（Special tokens）

有些字符串用于表示元信息，例如：

- `<|endoftext|>`

编码文本时，我们通常希望这些字符串始终作为一个**不可再拆分**的 token 保留。

例如 `<|endoftext|>` 应总是对应一个单独的 token ID，这样我们才能知道模型何时该停止生成。

这些 special tokens 必须加入词表中，并拥有固定 token ID。

示例（bpe_example）：BPE 训练示例

参考 Sennrich 等人 [2016] 的一个风格化例子。假设语料如下：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

并且词表中还有一个特殊 token `<|endoftext|>` 。

词表初始化

词表初始包含：

- `<|endoftext|>`
- 256 个单字节值

预分词

为了聚焦 merge 过程，这个示例里假设预分词只按空白切分。
统计后得到频率表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

把它表示成 `dict[tuple[bytes], int]` 会更方便，例如：

```
{(l,o,w): 5, ...}
```

注意：

- Python 中即使单个字节也是 `bytes` 对象
- Python 并没有专门的 `byte` 类型，就像也没有单独的 `char` 类型

Merge 过程

先统计所有相邻字节对的出现频次：

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
```

其中：

- `('es')`
- `('st')`

并列最高，因此按字典序选更大的 `('st')`。

随后不断执行 merge，最终会得到如下 merge 序列：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r'
```

如果只取前 6 次 merge，则得到：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
```

此时词表新增的元素包括：

```
[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]
```

于是单词 `newest` 会被编码成：

```
[ne, west]
```

2.5 进行 BPE 训练实验

现在请在 TinyStories 数据集上训练一个字节级 BPE 分词器。开始前建议先浏览一下数据集内容。

并行化预分词

你会发现主要瓶颈之一是预分词步骤。
你可以通过 Python 内置的 `multiprocessing` 对其并行化。

推荐做法：

- 把语料切成多个 chunk
- 保证 chunk 边界出现在 special token 开始处

你可以直接使用 starter code 中给出的边界获取逻辑：

`https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py`

之所以这样切块是安全的，是因为：

- 我们本来就不希望跨文档边界 merge

对于本作业，始终可以用这种方式分块。无需担心极端情况，例如超大语料里没有 `<|endoftext|>` 。

在预分词前移除特殊 token

在使用正则模式执行预分词（ `re.finditer` ）之前，应当先把语料中的 special token 拆开。

例如对于：

```
[Doc 1]<|endoftext|>[Doc 2]
```

应先按 `<|endoftext|>` 分开，再分别对 `[Doc 1]` 和 `[Doc 2]` 预分词。
这样就不会跨文档边界产生 merge。

可以借助 `re.split` 和 `"|".join(special_tokens)` 来实现，并注意使用 `re.escape` 。

测试 `test_train_bpe_special_tokens` 会检查这一点。

优化 merge 步骤

朴素 BPE 实现很慢，因为每次 merge 都要遍历所有 pair 频率。
实际上，每次 merge 后真正发生变化的只有那些与被 merge pair 重叠的 pair。

因此，可以通过：

- 建立 pair 计数索引
- 增量更新计数

来大幅加速训练。

注意：

- 这部分在 Python 中基本不可并行
- 但缓存与增量更新能显著提升速度

低资源提示：性能分析

请使用诸如 `cProfile` 或 `scalene` 的 profiling 工具，找出实现中的瓶颈，再集中优化它们。

低资源提示：降规模

建议不要一开始就在完整 TinyStories 上训练 tokenizer。
你可以先用一个较小“调试数据集”来验证实现，例如直接用 TinyStories 的验证集（22K 文档，而不是 212 万文档）。

这体现了一种通用策略：

- 能缩小规模时就先缩小
- 例如更小的数据集、更小的模型

但要注意：

- 调试集要足够大，能体现和完整配置相同的瓶颈
- 又不能大到每次跑都很久

题目（train_bpe）：BPE 分词器训练（15 分）

交付内容：编写一个函数，给定输入文本文件路径，训练一个字节级 BPE 分词器。

你的训练函数至少应支持以下参数：

- `input_path: str`
  训练数据文本文件路径
- `vocab_size: int`
  最终词表大小上限（包含初始字节词表、merge 新产生的词表项和 special token）
- `special_tokens: list[str]`
  要加入词表的特殊 token 列表。它们本身不改变 BPE 训练逻辑

训练函数应返回：

- `vocab: dict[int, bytes]`
  分词器词表，从 token ID 映射到 token 对应字节串
- `merges: list[tuple[bytes, bytes]]`
  BPE merge 列表，按创建顺序排列

要通过测试：

1. 先实现适配器 `adapters.run_train_bpe`
2. 运行：

```
uv run pytest tests/test_train_bpe.py
```

你的实现应通过全部测试。

可选项：如果你愿意投入较多时间，也可以把训练中的关键部分用系统语言实现，例如：

- C++（可考虑 `cppyy`）
- Rust（可考虑 `Py03`）

如果你这么做，请注意：

- 哪些操作会复制 Python 内存
- 哪些可以直接读取
- 需要留下清晰的构建说明

另外，GPT-2 使用的正则在很多 regex 引擎中支持不好或速度很慢。
我们验证过：

- Oniguruma：速度还可以，且支持 negative lookahead
- Python 的 `regex` 包：甚至往往更快

题目（train_bpe_tinystories）：在 TinyStories 上训练 BPE（2 分）

(a) 在 TinyStories 上训练一个字节级 BPE 分词器，最大词表大小设为 10,000，并把 TinyStories 的 `<|endoftext|>` special token 加入词表。
把得到的词表与 merges 序列化到磁盘。
训练花了多少小时、多少内存？词表中最长的 token 是什么？它合理吗？

资源要求：

- 不超过 30 分钟（无需 GPU）
- 不超过 30GB RAM

提示：如果你在预分词阶段使用多进程，并利用以下两点，BPE 训练应能做到 2 分钟以内：

1. `<|endoftext|>` 在数据中用于分隔文档
2. `<|endoftext|>` 在 BPE merge 应用前被作为 special case 处理

交付内容：1 到 2 句话回答。

(b) 对你的代码做 profiling。分词器训练过程中最耗时的是哪一部分？

交付内容：1 到 2 句话回答。

题目（train_bpe_expts_owt）：在 OpenWebText 上训练 BPE（2 分）

(a) 在 OpenWebText 上训练一个字节级 BPE 分词器，最大词表大小设为 32,000。
把词表与 merges 序列化到磁盘。
词表中最长的 token 是什么？它合理吗？

资源要求：

- 不超过 12 小时（无需 GPU）
- 不超过 100GB RAM

交付内容：1 到 2 句话回答。

(b) 比较在 TinyStories 与 OpenWebText 上训练得到的分词器。

交付内容：1 到 2 句话回答。

2.6 BPE 分词器：编码与解码

上一部分里，你已经实现了训练 BPE 分词器以得到：

- 词表
- merge 列表

现在，你要实现一个 BPE 分词器类，它能：

- 从给定词表和 merge 列表中加载
- 把文本编码为 token ID
- 或把 token ID 解码回文本

2.6.1 编码文本

BPE 的编码过程和训练词表时非常相似：

步骤 1：预分词

首先对输入文本进行预分词，并把每个 pre-token 表示成 UTF-8 字节序列。

步骤 2：按训练时顺序应用 merge

然后按照训练阶段生成 merge 的顺序，把这些 merge 应用于每个 pre-token。

示例（bpe_encoding）：BPE 编码示例

假设输入字符串是：

```
'the cat ate'
```

词表是：

```
{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b
```

学到的 merge 是：

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]
```

预分词后得到：

```
['the', ' cat', ' ate']
```

接着分别对每个 pre-token 应用 merge。

例如 `'the'`：

```
[b't', b'h', b'e'] -> [b'th', b'e'] -> [b'the']
```

于是其整数序列为：

```
[9]
```

继续处理 `' cat'` 与 `' ate'`，最终可得到：

```
[9, 7, 1, 5, 10, 3]
```

Special tokens

你的 tokenizer 在编码时应能正确处理用户提供的 special tokens。

内存考虑

如果你要 tokenize 一个大到无法一次性装入内存的文本文件，就需要：

- 把数据流切成可处理的小块
- 逐块处理

同时还要保证：

> token 不能跨 chunk 边界

否则，与“整段一次性 tokenize”的结果会不同。

2.6.2 解码文本

把整数 token ID 解码为原始文本时，做法很直接：

1. 查词表，把每个 ID 映射回字节串
2. 拼接成一个完整字节序列
3. 按 UTF-8 解码成 Unicode 字符串

注意：

- 输入 ID 未必一定能还原成合法 Unicode 文本

如果解码失败，你应当用官方 Unicode replacement character `U+FFFD` 替换非法字节。
在 Python 中，可以通过：

```
errors='replace'
```

来实现这一点。

题目（tokenizer）：实现分词器（15 分）

交付内容：实现一个 `Tokenizer` 类，根据给定词表和 merge 列表完成编码与解码。
还应支持用户提供的 special tokens（若不在词表中，应追加到词表）。

推荐接口：

```
def __init__(self, vocab, merges, special_tokens=None)
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
def encode(self, text: str) -> list[int]
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
def decode(self, ids: list[int]) -> str
```

测试步骤：

1. 实现适配器 `adapters.get_tokenizer`
2. 运行：

```
uv run pytest tests/test_tokenizer.py
```

你的实现应通过全部测试。

2.7 实验

题目（tokenizer_experiments）：分词器实验（4 分）

(a) 从 TinyStories 和 OpenWebText 各采样 10 篇文档。
分别使用你之前训练好的 TinyStories tokenizer（10K 词表）和 OpenWebText tokenizer（32K 词表）把这些文档编码成整数 ID。
每个 tokenizer 的压缩率（bytes/token）是多少？

交付内容：1 到 2 句话回答。

(b) 如果你用 TinyStories tokenizer 去 tokenize OpenWebText 样本，会发生什么？
比较压缩率，或定性描述结果。

交付内容：1 到 2 句话回答。

(c) 估算你的 tokenizer 吞吐量（例如 bytes/second）。
如果要 tokenize 825GB 文本的 Pile 数据集，需要多久？

交付内容：1 到 2 句话回答。

(d) 使用你的 TinyStories 和 OpenWebText tokenizer，对各自训练集和开发集编码成整数 token ID 序列。建议把 token ID 序列序列化成 `uint16` 类型的 NumPy 数组。为什么 `uint16` 是合适的选择？

交付内容：1 到 2 句话回答。

3. Transformer 语言模型架构

语言模型的输入是：

- 一批 token ID 序列
- 形状为 `(batch_size, sequence_length)`

输出是：

- 对每个位置预测“下一个 token”的归一化概率分布
- 形状为 `(batch_size, sequence_length, vocab_size)`

训练时：

- 我们会用这些 next-token predictions 计算交叉熵损失

推理生成时：

- 取最后一个位置的 next-token 分布
- 决定下一个 token
- 把生成出的 token 拼回输入
- 重复这个过程

在本部分，你将从零实现这个 Transformer LM。

3.1 Transformer LM

给定一串 token ID，Transformer LM 的流程是：

1. 用输入 embedding 把 token ID 转成稠密向量
2. 经过 `num_layers` 个 Transformer block
3. 再通过一个线性投影（output embedding / LM head）得到 next-token logits

3.1.1 Token Embeddings

Embedding 层输入：

- 形状 `(batch_size, sequence_length)` 的整数 token ID tensor

输出：

- 形状 `(batch_size, sequence_length, d_model)` 的向量序列

3.1.2 Pre-norm Transformer Block

一个标准 decoder-only Transformer LM 包含 `num_layers` 个同构的 Transformer block。
每个 block：

- 输入形状 `(batch_size, sequence_length, d_model)`
- 输出形状仍为 `(batch_size, sequence_length, d_model)`

block 通过两类操作处理序列：

- self-attention：在序列范围内聚合信息
- feed-forward：施加逐位置的非线性变换

3.2 输出归一化与输出 embedding

经过 `num_layers` 个 Transformer block 后，我们需要把最终激活变成对词表的分布。

由于本作业实现的是 **pre-norm Transformer block**，所以在最后一个 block 后还需要再做一次 layer normalization，以保证输出尺度合理。

随后使用一个标准的线性层，把最终隐藏状态映射成 next-token logits。

3.3 备注：批处理、Einsum 与高效计算

Transformer 中的大量计算，实际上都是对很多“类似 batch 的维度”重复执行相同操作。

例如：

- batch 维
- sequence length 维中的逐位置操作
- 多头注意力中的 head 维

在 PyTorch 中，很多操作都支持额外的前置 batch-like 维度，并能高效广播。

例如，如果：

- 数据 tensor `D` 的形状是 `(batch_size, sequence_length, d_model)`
- 矩阵 `A` 的形状是 `(d_model, d_model)`

那么：

```
D @ A
```

就会执行 batched matrix multiply。

不过，如果你手动用 `view` 、 `reshape` 、 `transpose` 来整理维度，可读性会变差。
因此推荐使用：

- `torch.einsum`
- `einops`
- 或更通用的 `einx`

这类写法更具自解释性，也更灵活。

我们**强烈建议**你为本课程学习并使用 einsum 记号。

3.3.1 数学记号与内存顺序

许多机器学习论文使用行向量记号，这与 NumPy/PyTorch 默认的 row-major 内存顺序比较匹配。
而本作业中的数学推导使用列向量记号，因为更利于阅读和理解推导。

如果你直接使用矩阵乘法记号，需要记得：

- PyTorch 内部更自然的是 row-major 约定

如果你主要用 einsum 来写矩阵运算，这个问题基本可以忽略。

3.4 基本构件：Linear 与 Embedding 模块

3.4.1 参数初始化

神经网络训练往往依赖合理初始化。坏初始化可能导致：

- 梯度消失
- 梯度爆炸

Pre-norm Transformer 对初始化相对更鲁棒，但初始化仍会显著影响训练速度与收敛。

本作业中先使用以下近似初始化：

- Linear 权重：截断高斯分布，均值 0，方差约为 `2 / (d_in + d_out)` ，截断到 `[−3σ, 3σ]`
- Embedding：截断高斯分布，均值 0，方差 1，截断到 `[−3, 3]`
- RMSNorm：初始化为 1

请使用 `torch.nn.init.trunc_normal_` 。

3.4.2 Linear 模块

Linear 层是 Transformer 与一般神经网络的核心构件之一。
你需要实现一个自己的 `Linear` 类，继承 `torch.nn.Module` ，执行如下线性变换：

$$y = Wx$$

注意：我们**不使用 bias**，这与多数现代 LLM 一致。

题目（linear）：实现线性层（1 分）

交付内容：实现一个 `Linear` 类，行为类似 PyTorch 的 `nn.Linear` ，但不包含 bias 参数。

推荐接口：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
def forward(self, x: torch.Tensor) -> torch.Tensor
```

要求：

- 继承 `nn.Module`
- 调用父类构造函数
- 参数以 `W` （不是 `W^T` ）的形式存储，且用 `nn.Parameter` 包装
- 不能用 `nn.Linear` 或 `nn.functional.linear`

测试：

1. 实现适配器 `adapters.run_linear`
2. 运行：

```
uv run pytest -k test_linear
```

3.4.3 Embedding 模块

Embedding 层把 token ID 映射为 `d_model` 维向量。

你需要实现一个自定义 `Embedding` 类，继承 `torch.nn.Module` ，而不能使用 `nn.Embedding` 。

题目（embedding）：实现 embedding 模块（1 分）

推荐接口：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

要求：

- 继承 `nn.Module`
- 调用父类构造函数
- embedding 矩阵用 `nn.Parameter` 保存
- `d_model` 作为 embedding 矩阵的最后一维
- 不使用 `nn.Embedding` 或 `nn.functional.embedding`

测试：

1. 实现适配器 `adapters.run_embedding`
2. 运行：

```
uv run pytest -k test_embedding
```

3.5 Pre-norm Transformer Block

每个 Transformer block 含有两个子层：

- multi-head self-attention
- position-wise feed-forward

原始 Transformer 使用的是：

- 残差连接
- 子层后做 layer normalization

这叫 **post-norm**

而大量后续工作发现，把 normalization 移到子层输入处，并在整个网络最后再加一次 norm，会更稳定，这就是 **pre-norm**。

一种直观理解是：

- 从输入 embedding 到最终输出之间，存在一条更“干净”的 residual stream
- 更利于梯度流动

今天几乎所有主流语言模型都采用 pre-norm（例如 GPT-3、LLaMA、PaLM）。

3.5.1 RMSNorm

原始 Transformer 使用 LayerNorm。
本作业中，参考 Touvron 等人 [2023]，采用 RMSNorm。

给定激活向量 $a \in R^{d_{model}}$，RMSNorm 定义为：

$$\mathrm{RMSNorm}(a_i) = \frac{a_i}{\mathrm{RMS}(a)} g_i$$

其中：

$$\mathrm{RMS}(a) = \sqrt{\frac{1}{d_{model}} \sum_i a_i^2 + \epsilon}$$

为了避免平方时溢出，你应在 forward 中先把输入 upcast 到 `torch.float32`，再在最后 cast 回原 dtype。

题目（rmsnorm）：实现 RMSNorm（1 分）

推荐接口：

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
def forward(self, x: torch.Tensor) -> torch.Tensor
```

测试：

1. 实现适配器 `adapters.run_rmsnorm`
2. 运行：

```
uv run pytest -k test_rmsnorm
```

3.5.2 Position-Wise Feed-Forward Network

原始 Transformer 的 FFN 是：

- 两层线性层
- 中间一个 ReLU

现代大模型通常做了两点修改：

1. 换了激活函数
2. 加入了 gating 机制

本作业中，你将实现 **SwiGLU**，它结合了：

- SiLU / Swish 激活
- GLU（Gated Linear Unit）

SiLU 定义为：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

GLU 定义为：

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x$$

SwiGLU 则为：

$$\mathrm{FFN}(x) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x)$$

其中：

- $x \in R^{d_{model}}$
- $W_1, W_3 \in R^{d_{ff} \times d_{model}}$
- $W_2 \in R^{d_{model} \times d_{ff}}$
- 通常 $d_{ff} = \frac{8}{3} d_{model}$

题目（positionwise_feedforward）：实现逐位置前馈网络（2 分）

交付内容：实现 SwiGLU 前馈网络。

说明：

- 你可以直接用 `torch.sigmoid`，以保证数值稳定性
- 实现中应设置 `d_ff ≈ 8/3 * d_model`
- 同时保证 `d_ff` 是 64 的倍数，以充分利用硬件

测试：

1. 实现适配器 `adapters.run_swiglu`
2. 运行：

```
uv run pytest -k test_swiglu
```

3.5.3 相对位置嵌入：RoPE

为了给模型注入位置信息，本作业中使用 **Rotary Position Embeddings（RoPE）** [Su et al., 2021]。

对于位置 i 上的 query：

$$q^{(i)} = W_q x^{(i)}$$

我们对它施加一个成对旋转矩阵 R_i，得到：

$$q'^{(i)} = R_i q^{(i)}$$

同样也对 key 做旋转。

注意：

- 旋转作用于 query 和 key
- **不作用于 value**

如果你想进一步优化，可以：

- 在 `__init__` 时预计算 sin/cos buffer
- 用 `self.register_buffer(persistent=False)` 保存

题目（rope）：实现 RoPE（2 分）

推荐接口：

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

要求：

- `x` 的 shape 为 `(..., seq_len, d_k)`
- 支持任意数量 batch-like 维度
- `token_positions` 的 shape 为 `(..., seq_len)`
- 使用 token positions 从预计算的 sin/cos 张量中切片

测试：

1. 实现适配器 `adapters.run_rope`
2. 运行：

```
uv run pytest -k test_rope
```

3.5.4 Scaled Dot-Product Attention

我们实现 Vaswani 等人 [2017] 中的 scaled dot-product attention：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

在此之前，你需要先实现 softmax。

softmax 定义为：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_j \exp(v_j)}$$

为了数值稳定，通常会先减去最大值。

题目（softmax）：实现 softmax（1 分）

交付内容：实现一个 softmax 函数，接受：

- 一个 tensor
- 一个维度 `i`

并沿该维度应用 softmax。

要求：

- 使用“先减去该维度最大值”的技巧

测试：

1. 实现适配器 `adapters.run_softmax`
2. 运行：

```
uv run pytest -k test_softmax_matches_pytorch
```

Masking

有时我们希望 mask attention 输出。
mask 的形状为：

$$M \in \{True, False\}^{n \times m}$$

约定：

- `True` 表示 query `i` 可以 attend 到 key `j`
- `False` 表示不能 attend

计算上，一般做法是：

- 对 mask 为 `False` 的位置加上 `-∞`
- 再做 softmax

题目（scaled_dot_product_attention）：实现缩放点积注意力（5 分）

交付内容：实现 scaled dot-product attention。

要求：

- keys / queries 的 shape： `(batch_size, ..., seq_len, d_k)`
- values 的 shape： `(batch_size, ..., seq_len, d_v)`
- 返回输出 shape： `(batch_size, ..., d_v)` （按你实现中最终 attention 聚合后的格式）
- 支持可选布尔 mask，形状为 `(seq_len, seq_len)`
- mask 为 `True` 的位置，其注意力概率归一化后总和应为 1
- mask 为 `False` 的位置，其注意力概率应为 0

测试：

1. 实现适配器 `adapters.run_scaled_dot_product_attention`
2. 运行：

```
uv run pytest -k test_scaled_dot_product_attention
uv run pytest -k test_4d_scaled_dot_product_attention
```

3.5.5 因果多头自注意力

多头注意力定义如下：

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(head_1, \ldots, head_h)$$

其中：

$$head_i = \mathrm{Attention}(Q_i, K_i, V_i)$$

因此多头自注意力为：

$$\text{MultiHeadSelfAttention}(x) = W_O \cdot \text{MultiHead}(W_Q x, W_K x, W_V x)$$

因果 masking

你的实现应阻止模型看到未来 token。
也就是说：

- 位置 `i` 只能 attend 到 `j <= i`

你可以使用：

- `torch.triu`
- 或广播式索引比较

来构造这个因果 mask。

应用 RoPE

RoPE 只作用于：

- query
- key

而不作用于 value。

同时，多头中的 head 维应被当作 batch-like 维处理。

题目（multihead_self_attention）：实现因果多头自注意力（5 分）

交付内容：实现一个 `torch.nn.Module` 版本的 causal multi-head self-attention。

至少支持以下参数：

- `d_model: int`
- `num_heads: int`

并按照 Vaswani 等人 [2017] 的设置：

- $d_k = d_v = d_{model}/h$

测试：

1. 实现适配器 `adapters.run_multihead_self_attention`
2. 运行：

```
uv run pytest -k test_multihead_self_attention
```

3.6 完整 Transformer LM

现在开始把各个模块组装起来。

一个 Transformer block 包含两个 sublayer：

1. multi-head self-attention
2. feed-forward network

在每个 sublayer 中：

1. 先做 RMSNorm
2. 再执行主要操作（MHA / FFN）
3. 最后加上 residual connection

例如第一个子层可写为：

$$y = x + \mathrm{MultiHeadSelfAttention}(\mathrm{RMSNorm}(x))$$

题目（transformer_block）：实现 Transformer block（3 分）

交付内容：实现 pre-norm Transformer block。

至少支持：

- `d_model: int`
- `num_heads: int`
- `d_ff: int`

测试：

1. 实现适配器 `adapters.run_transformer_block`
2. 运行：

```
uv run pytest -k test_transformer_block
```

题目（transformer_lm）：实现 Transformer LM（3 分）

把所有模块组装成完整的 Transformer 语言模型。
除 Transformer block 的参数外，还至少支持：

- `vocab_size: int`
- `context_length: int`
- `num_layers: int`

测试：

1. 实现适配器 `adapters.run_transformer_lm`
2. 运行：

```
uv run pytest -k test_transformer_lm
```

交付内容：一个能够通过上述测试的 Transformer LM 模块。

资源核算：Transformer FLOPs

理解 Transformer 各部分如何消耗算力与内存非常重要。
本题中你将做基础 FLOPs accounting。

基本规则：

> 若 $A \in R^{m \times n}$，$B \in R^{n \times p}$，则矩阵乘法 AB 需要 $2mnp$ FLOPs。

题目（transformer_accounting）：Transformer LM 资源核算（5 分）

(a) 考虑 GPT-2 XL，配置如下：

- `vocab_size: 50,257`
- `context_length: 1,024`
- `num_layers: 48`
- `d_model: 1,600`
- `num_heads: 25`
- `d_ff: 6,400`

如果按该配置构建模型，它一共有多少可训练参数？
若每个参数用单精度浮点表示，仅加载模型需要多少内存？

交付内容：1 到 2 句话回答。

(b) 列出一次 forward pass 中所有矩阵乘法，以及各自 FLOPs，总 FLOPs 是多少？
假设输入序列长度为 `context_length` 。

交付内容：矩阵乘法列表（附说明）以及总 FLOPs。

(c) 根据上述分析，模型中哪些部分 FLOPs 最多？

交付内容：1 到 2 句话回答。

(d) 对 GPT-2 small、medium、large 重复上述分析。随着模型增大，哪些模块的 FLOPs 占比上升或下降？

交付内容：对每个模型给出各组件 FLOPs 占比，并说明不同模型规模下趋势如何变化。

(e) 把 GPT-2 XL 的 context length 提升到 16,384。
一次 forward pass 的总 FLOPs 如何变化？各模块 FLOPs 相对占比如何变化？

交付内容：1 到 2 句话回答。

4. 训练 Transformer LM

现在你已经有了：

- 数据预处理（tokenizer）
- 模型（Transformer）

还剩训练支持代码：

- 损失函数：cross-entropy
- 优化器：AdamW
- 训练循环：加载数据、保存 checkpoint、管理训练过程

4.1 交叉熵损失

回忆 Transformer LM 对每个序列 x 定义了条件分布：

$$p_\theta(x_{i+1} \mid x_{1:i})$$

标准交叉熵（负对数似然）为：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1}|x_{1:i})$$

Transformer 会在每个位置输出 logits $o_i \in R^{vocab_size}$，于是：

$$p(x_{i+1}|x_{1:i}) = \mathrm{softmax}(o_i)[x_{i+1}]$$

实现交叉熵时同样要注意数值稳定性。

题目（cross_entropy）：实现交叉熵（1 分）

交付内容：实现一个函数，输入：

- 预测 logits o_i
- 目标标签 x_{i+1}

计算：

$$\ell_i = -\log \mathrm{softmax}(o_i)[x_{i+1}]$$

要求：

- 先减最大值以提高数值稳定性
- 能约掉的 `log` 与 `exp` 要尽量约掉
- 支持额外 batch-like 维度，并返回 batch 平均值

测试：

1. 实现适配器 `adapters.run_cross_entropy`
2. 运行：

```
uv run pytest -k test_cross_entropy
```

困惑度（Perplexity）

训练时只需要交叉熵，但评估时通常还要报告 perplexity：

$$\text{perplexity} = \exp\left(\frac{1}{m}\sum_{i=1}^{m}\ell_i\right)$$

4.2 SGD 优化器

最简单的基于梯度的优化器是随机梯度下降（SGD）：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t)$$

其中：

- B_t 是当前 batch
- α_t 是学习率

4.2.1 在 PyTorch 中实现 SGD

你需要继承 `torch.optim.Optimizer` 。一个优化器子类至少要实现两个方法：

- `__init__`
- `step`

`step` 中通常要：

- 遍历每个参数
- 读取其梯度 `p.grad`
- 原地更新 `p.data`

handout 中给了一个带学习率衰减的 SGD 示例：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}}\nabla L(\theta_t; B_t)$$

并给出了完整 PyTorch 优化器代码示例。

题目（learning_rate_tuning）：调学习率（1 分）

在 handout 给出的 SGD toy 示例中，把学习率改为：

- `1e1`

- `1e2`
- `1e3`

各跑 10 个训练迭代。
观察 loss 是：

- 衰减更快
- 衰减更慢
- 还是发散

交付内容：1 到 2 句话说明你的观察结果。

4.3 AdamW

现代语言模型几乎都不再用纯 SGD，而是使用 Adam 系列优化器。
本作业中使用的是 **AdamW** [Loshchilov and Hutter, 2019]。

AdamW 相比 Adam 的关键改进是：

- 以与梯度更新解耦的方式加入 weight decay

AdamW 是**有状态（stateful）**的，每个参数需要维护：

- 一阶矩估计
- 二阶矩估计

因此它会消耗更多内存，但通常换来更好的稳定性与收敛。

超参数包括：

- 学习率 α
- (β_1, β_2)
- ϵ
- weight decay λ

常见设置是：

- 一般：`(0.9, 0.999)`
- 大模型常见：`(0.9, 0.95)`

题目（adamw）：实现 AdamW（2 分）

交付内容：实现一个继承自 `torch.optim.Optimizer` 的 AdamW。

应在 `__init__` 中接收：

- 学习率 α
- β

- ϵ
- λ

你可以使用 `self.state` 来维护参数状态。

测试：

1. 实现适配器 `adapters.get_adamw_cls`
2. 运行：

```
uv run pytest -k test_adamw
```

题目（adamwAccounting）：AdamW 训练资源核算（2 分）

假设所有 tensor 都使用 `float32` 。

(a) AdamW 训练的峰值内存是多少？请分别从以下角度分解：

- 参数
- 激活
- 梯度
- 优化器状态

请用：

- `batch_size`
- `vocab_size`
- `context_length`
- `num_layers`
- `d_model`
- `num_heads`

表示，并假设：

- `d_ff = 4 * d_model`

为了简化激活内存计算，只考虑：

- Transformer block
  - RMSNorm
  - MHA 子层：QKV projection、QK^T、softmax、加权求和、输出投影
  - FFN：W1、SiLU、W2
- final RMSNorm
- output embedding
- logits 上的 cross-entropy

交付内容：分别给出参数、激活、梯度、优化器状态及总和的代数表达式。

(b) 对 GPT-2 XL 代入，得到只依赖 `batch_size` 的表达式。
在 80GB 显存下，最大 batch size 是多少？

交付内容：形如 `a * batch_size + b` 的表达式以及最大 batch size。

(c) 一步 AdamW 需要多少 FLOPs？

交付内容：代数表达式与简短说明。

(d) 假设 A100 的 float32 峰值是 19.5 TFLOP/s，且你能达到 50% MFU。
在单张 A100 上，以 batch size 1024 训练 400K steps 的 GPT-2 XL 需要多少天？
假设 backward FLOPs 是 forward 的两倍。

交付内容：训练所需天数与简短说明。

4.4 学习率调度

训练过程中，最佳学习率通常不是固定不变的。
在 Transformer 训练中，常见做法是：

- 先 warmup
- 再逐渐衰减

本作业要求实现 LLaMA 中使用的 cosine annealing schedule。

它需要以下参数：

- 当前步数 `t`
- 最大学习率 `α_max`
- 最小学习率 `α_min`
- warmup 步数 `T_w`
- cosine 衰减步数 `T_c`

调度规则：

- Warm-up：若 `t < T_w`，则 `α_t = (t / T_w) * α_max`
- Cosine annealing：若 `T_w <= t <= T_c`
- Post-annealing：若 `t > T_c`，则 `α_t = α_min`

题目（learning_rate_schedule）：实现带 warmup 的余弦学习率调度

交付内容：实现一个函数，根据上述规则返回第 `t` 步的学习率。
然后实现：

- `adapters.get_lr_cosine_schedule`

并运行：

```
uv run pytest -k test_get_lr_cosine_schedule
```

4.5 梯度裁剪

训练中有时会遇到梯度特别大的样本，导致训练不稳定。
常见缓解方法是梯度裁剪（gradient clipping）。

给定所有参数梯度组成的向量 g，计算其 ℓ_2 范数：

$$\|g\|_2$$

若它小于阈值 M，则不变；否则按比例缩放：

$$g \leftarrow g \cdot \frac{M}{\|g\|_2 + \epsilon}$$

本作业使用：

- $\epsilon = 10^{-6}$

题目（gradient_clipping）：实现梯度裁剪（1 分）

交付内容：实现一个函数，输入：

- 参数列表
- 最大 ℓ_2 范数

并原地修改每个参数的梯度。

测试：

1. 实现适配器 `adapters.run_gradient_clipping`
2. 运行：

```
uv run pytest -k test_gradient_clipping
```

5. 训练循环

现在把前面的组件真正组合起来：

- tokenized data
- model
- optimizer

5.1 Data Loader

经过分词后的数据可以看作一长串 token：

$$x = (x_1, \ldots, x_n)$$

尽管原始语料可能是许多独立文档，训练时通常会把它们串接在一起，并在文档之间插入一个分隔符，例如 `<|endoftext|>` 。

Data loader 负责把这串 token 转成 batch 流。
每个 batch 包含：

- `B` 个长度为 `m` 的输入序列
- 对应长度为 `m` 的 next-token 目标序列

例如当：

- `B = 1`
- `m = 3`

时，一个 batch 可能是：

```
([x2, x3, x4], [x3, x4, x5])
```

这种做法有几个优点：

- 任意 `1 <= i < n - m` 都能形成合法训练样本
- 所有训练序列长度一致，无需 padding
- 无需把完整数据集一次性读入内存

题目（data_loading）：实现数据加载（2 分）

交付内容：写一个函数，输入：

- 一个 NumPy 整数数组 `x` （token IDs）
- `batch_size`
- `context_length`
- PyTorch device 字符串（如 `'cpu'` 或 `'cuda:0'` ）

返回：

- 输入 tensor
- 对应目标 tensor

两者形状都应为：

```
(batch_size, context_length)
```

并放在指定 device 上。

测试：

1. 实现适配器 `adapters.run_get_batch`
2. 运行：

```
uv run pytest -k test_get_batch
```

低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果你在 CPU 或 Apple Silicon 上训练：

- 数据要移到与模型一致的 device

例如：

- CPU： `'cpu'`
- Apple Silicon： `'mps'`

如果数据集太大放不进内存怎么办？

可以使用 `mmap`：

- 把磁盘文件映射进虚拟内存
- 按需加载

在 NumPy 中可以通过：

- `np.memmap`
- 或 `np.load(..., mmap_mode='r')`

来实现。

在训练时，请务必用 memory-mapped 的方式加载大数据集。

5.2 Checkpointing

训练过程中，我们需要定期保存 checkpoint，以便：

- 作业中断后恢复
- 分析中间阶段模型
- 研究训练动态

一个 checkpoint 至少应包含：

- 模型参数
- 优化器状态

- 当前迭代步数

PyTorch 提供了现成支持：

- `state_dict()`
- `load_state_dict()`
- `torch.save`
- `torch.load`

题目（checkpointing）：实现模型 checkpoint（1 分）

实现两个函数：

```
save_checkpoint(model, optimizer, iteration, out)
load_checkpoint(src, model, optimizer)
```

要求：

- `save_checkpoint` 保存模型、优化器和迭代步数
- `load_checkpoint` 恢复它们，并返回保存时的步数

测试：

1. 实现适配器 `adapters.run_save_checkpoint` 与 `adapters.run_load_checkpoint`
2. 运行：

```
uv run pytest -k test_checkpointing
```

5.3 训练循环

现在终于可以把所有组件组合到主训练脚本中了。
建议让你的训练脚本易于更换超参数，例如通过命令行参数传入。

题目（training_together）：把一切组合起来（4 分）

交付内容：写一个训练脚本，至少支持：

- 配置模型和优化器超参数
- 使用 `np.memmap` 高效加载训练/验证数据
- 把 checkpoint 保存到用户指定路径
- 周期性记录训练和验证性能（例如输出到控制台或 Weights & Biases）

6. 生成文本

现在我们已经可以训练模型，最后一步是让模型生成文本。

语言模型输出的是 logits，因此在采样前还需要：

- 先通过 softmax 转成概率分布

Decoding

给定 prompt（前缀 token 序列），模型会预测下一 token 的词表分布。
然后从该分布中采样一个 token，并把它追加到输入中，反复迭代，直到：

- 生成 `<|endoftext|>`
- 或达到最大生成长度

Decoder tricks

小模型容易生成质量很差的文本。这里介绍两个常见 trick。

1. Temperature scaling

$$\mathrm{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_j \exp(v_j/\tau)}$$

当 $\tau \to 0$ 时，分布会越来越接近 one-hot，即更偏向选最大项。

2. Top-p / Nucleus sampling

给定概率分布 `q`，top-p 会截断低概率词，只保留最小集合 `V(p)`，使得：

$$\sum_{j \in V(p)} q_j \geq p$$

然后只在该集合中重新归一化并采样。

题目（decoding）：解码（3 分）

交付内容：实现一个解码函数，建议至少支持：

- 根据用户给出的 prompt 生成补全文本
- 用户可控制最大生成 token 数
- 支持 temperature scaling
- 支持 top-p / nucleus sampling

7. 实验

现在你已经把系统搭起来了，接下来要真正训练（小型）语言模型并做实验。

7.1 如何做实验与交付内容

理解 Transformer 架构最好的方式就是：

> 自己改它、自己跑它

因此你要能够：

- 快速做实验
- 稳定复现实验
- 记录实验过程与曲线

为了支持提交 loss 曲线，请确保：

- 周期性评估验证集 loss
- 记录训练步数
- 记录 wallclock time

题目（experiment_log）：实验日志（3 分）

交付内容：

- 为训练与评估代码加入实验追踪基础设施
- 维护一份实验日志文档，记录本节下面所有题目中你做过的尝试

7.2 TinyStories

我们先从一个非常简单的数据集开始：TinyStories。

它训练快，而且容易观察到一些有趣行为。

示例（tinystories_example）

原文给出了一段 TinyStories 的样本文本，展示其典型风格：简短、简单、儿童故事式英语。

推荐初始超参数

- `vocab_size = 10000`
- `context_length = 256`
- `d_model = 512`
- `d_ff = 1344`
- `RoPE theta = 10000`
- `num_layers = 4`

- `num_heads = 16`
- 总处理 token 数约为 `327,680,000`

你还需要自己调节：

- learning rate
- warmup
- AdamW 的其他超参数（`\beta_1, \beta_2, \epsilon`）
- weight decay

调试架构的一些建议

- 先尝试过拟合一个 minibatch
- 在各组件中打断点，检查中间张量 shape
- 监控激活、参数和梯度范数，观察是否爆炸或消失

题目（learning_rate）：调学习率（3 分，4 H100 小时）

(a) 对学习率做 sweep，并报告最终 loss 或是否发散。

交付内容：

- 多个学习率对应的 learning curves
- 说明你的超参数搜索策略

额外交付内容：

- 一个在 TinyStories 上验证 loss（per-token）不高于 1.45 的模型

低资源提示：在 CPU 或 Apple Silicon 上少步数训练

如果你在 `cpu` 或 `mps` 上跑：

- 可把总 token 数降到 `40,000,000`
- 验证 loss 目标从 1.45 放宽到 2.00

staff 参考实现中，在 M3 Max 36GB RAM 上：

- `32 × 5000 × 256 = 40,960,000` tokens
- CPU：1 小时 22 分
- MPS：36 分钟
- 第 5000 步时验证 loss 约为 1.80

额外建议：

- 如果总训练步数是 `x`，建议余弦退火在第 `x` 步恰好衰减到最小学习率
- 在 `mps` 上不要开启 TF32 kernels
- 可以尝试 `torch.compile` 提速

(b) 经验法则说“最佳学习率通常在稳定性边界附近”。
请研究发散点与最佳学习率之间的关系。

交付内容：

- 一组逐渐增大学习率的 learning curves，其中至少包含一次发散
- 分析它与收敛速度的关系

题目（batch_size_experiment）：batch size 变化实验（1 分，2 H100 小时）

把 batch size 从 1 一路调到显存上限，中间至少尝试几个值，例如：

- 64
- 128

交付内容：

- 不同 batch size 对应的 learning curves
- 如有必要重新调 learning rate
- 若干句讨论 batch size 对训练的影响

题目（generate）：生成文本（1 分）

使用你的 decoder 和训练好的 checkpoint，生成模型文本。
你可能需要调整：

- temperature
- top-p

交付内容：

- 至少 256 个 token 的生成文本（或直到第一个 `<|endoftext|>` ）
- 简要评论其流畅度
- 至少列出两个影响生成质量的因素

7.3 消融与架构修改

真正理解 Transformer 的好办法之一，是亲手删掉或替换某些组件，观察会发生什么。

题目（layer_norm_ablation）：移除 RMSNorm 并训练（1 分，1 H100 小时）

从 Transformer block 中移除所有 RMSNorm，再训练模型。

问题：

- 在之前的最佳学习率下会怎样？
- 降低学习率后能否恢复稳定？

交付内容：

- 去掉 RMSNorm 后的学习曲线
- 最佳学习率下的学习曲线
- 若干句评论 RMSNorm 的影响

题目（pre_norm_ablation）：实现 post-norm 并训练（1 分，1 H100 小时）

把 pre-norm 改成 post-norm，然后训练。

交付内容：

- post-norm 与 pre-norm 的学习曲线对比

题目（no_pos_emb）：实现 NoPE（1 分，1 H100 小时）

把 Transformer 中的 RoPE 完全移除，不加入任何位置编码信息，观察表现。

交付内容：

- RoPE 与 NoPE 的学习曲线对比

题目（swiglu_ablation）：SwiGLU vs. SiLU（1 分，1 H100 小时）

比较：

- 带 gating 的 SwiGLU
- 不带 GLU，只用 SiLU 的 FFN

注意：

- 为了参数量可比，SiLU 版本应设 `d_ff = 4 * d_model`

交付内容：

- SwiGLU 与 SiLU 的学习曲线对比
- 若干句讨论你的发现

低资源提示：GPU 资源有限的同学可继续在 TinyStories 上测试修改

后续我们将转向更大、更嘈杂的 OpenWebText。
如果你 GPU 资源有限，建议继续在 TinyStories 上测试各种修改，并以验证集 loss 作为评估指标。

7.4 在 OpenWebText 上运行

现在我们换到更标准的网页抓取预训练数据集 OpenWebText。

它比 TinyStories：

- 更真实
- 更复杂
- 风格更丰富

这也是为什么它更接近真实预训练任务。

题目（main_experiment）：在 OWT 上做主实验（2 分，3 H100 小时）

使用与 TinyStories 相同的模型架构与总训练迭代数，在 OpenWebText 上训练语言模型。

交付内容：

- OWT 上的 learning curve
- 说明它与 TinyStories 的 loss 有什么差异，以及如何理解这些 loss

额外交付内容：

- OpenWebText LM 的生成文本（与 TinyStories 输出格式一致）
- 评论它的流畅度
- 说明为什么在相同模型和算力预算下，它的输出质量仍然比 TinyStories 差

7.5 你自己的修改 + 排行榜

恭喜做到这里。最后一部分，你需要尝试改进 Transformer，并看看你的超参数与架构在班级中表现如何。

排行榜规则

限制只有以下两条：

运行时间
提交模型最多只能在单张 H100 上运行 1.5 小时。
你可以在 slurm 脚本里设置：

```
--time=01:30:00
```

数据
只能使用我们提供的 OpenWebText 训练集。

除此之外，你可以自由尝试。

如果你需要灵感，可以参考：

- Llama 3、Qwen 2.5 等开源 LLM 家族
- NanoGPT speedrun 仓库中的各种“速通预训练”修改

例如一个很常见的改动是：

- 输入 embedding 与输出 embedding 权重共享（weight tying）

如果尝试 weight tying，你可能需要减小 embedding / LM head 的初始化标准差。

建议你先在：

- TinyStories
- 或 OWT 的小子集

上验证修改，再做完整 1.5 小时训练。

题目（leaderboard）：排行榜（6 分，10 H100 小时）

在满足排行榜规则的前提下训练一个模型，目标是在 1.5 H100 小时内尽量降低验证 loss。

交付内容：

- 最终验证 loss
- 一条学习曲线，横轴必须是 wallclock time，且总时长小于 1.5 小时
- 描述你做了哪些改动

要求：

- 你的提交至少应优于 naive baseline： `loss = 5.0`

提交地址：

```
https://github.com/stanford-cs336/assignment1-basics-leaderboard
```

参考文献

以下参考文献保留原文英文形式：

- Ronen Eldan and Yuanzhi Li. *TinyStories: How small can language models be and still speak coherent English?* 2023.
- Aaron Gokaslan et al. *OpenWebText corpus*. 2019.
- Rico Sennrich et al. *Neural machine translation of rare words with subword units*. ACL 2016.
- Changhan Wang et al. *Neural machine translation with byte-level subwords*. 2019.
- Philip Gage. *A new algorithm for data compression*. 1994.
- Alec Radford et al. *Language models are unsupervised multitask learners*. 2019.
- Alec Radford et al. *Improving language understanding by generative pre-training*. 2018.
- Ashish Vaswani et al. *Attention is all you need*. NeurIPS 2017.
- Toan Q. Nguyen and Julian Salazar. *Transformers without tears*. 2019.
- Ruibin Xiong et al. *On layer normalization in the Transformer architecture*. 2020.
- Jimmy Lei Ba et al. *Layer normalization*. 2016.
- Hugo Touvron et al. *Llama*. 2023.

- Biao Zhang and Rico Sennrich. *Root mean square layer normalization*. 2019.
- Aaron Grattafiori et al. *The Llama 3 herd of models*. 2024.
- An Yang et al. *Qwen2.5 technical report*. 2024.
- Aakanksha Chowdhery et al. *PaLM*. 2022.
- Dan Hendrycks and Kevin Gimpel. *GELU*. 2016.
- Stefan Elfwing et al. *Sigmoid-weighted linear units*. 2017.
- Yann N. Dauphin et al. *Language modeling with gated convolutional networks*. 2017.
- Noam Shazeer. *GLU variants improve transformer*. 2020.
- Jianlin Su et al. *RoFormer*. 2021.
- Diederik P. Kingma and Jimmy Ba. *Adam*. 2015.
- Ilya Loshchilov and Frank Hutter. *Decoupled weight decay regularization*. 2019.
- Tom B. Brown et al. *Language models are few-shot learners*. 2020.
- Jared Kaplan et al. *Scaling laws for neural language models*. 2020.
- Jordan Hoffmann et al. *Training compute-optimal large language models*. 2022.
- Ari Holtzman et al. *The curious case of neural text degeneration*. 2020.
- Yao-Hung Hubert Tsai et al. *Transformer dissection*. 2019.
- Amirhossein Kazemnejad et al. *The impact of positional encoding on length generalization in transformers*. 2023.